
Containerization and DevOps Lab

EXPERIMENT – 01

Name: ARYAN RAJ
Batch: B4
SAP ID: 500126035
Roll No: R2142231908

1 Objective

- To understand the conceptual and practical differences between Virtual Machines Containers.
 - To install and configure a Virtual Machine using VirtualBox and Vagrant on Windows.
 - To install and configure Containers using Docker inside WSL.
 - To deploy an Ubuntu-based Nginx web server in both environments.
- To compare resource utilization, performance, and operational characteristics of VMs and Containers.

2 Theory

2.1 Virtual Machine (VM)

A Virtual Machine emulates a complete physical computer, including its own operating system kernel, hardware drivers, and user space. Each VM runs on top of a hypervisor.

Characteristics:

- Full OS per VM
- Higher resource usage
- Strong isolation
- Slower startup time

2.2 Container

Containers virtualize at the operating system level. They share the host OS kernel while isolating applications and dependencies in user space.

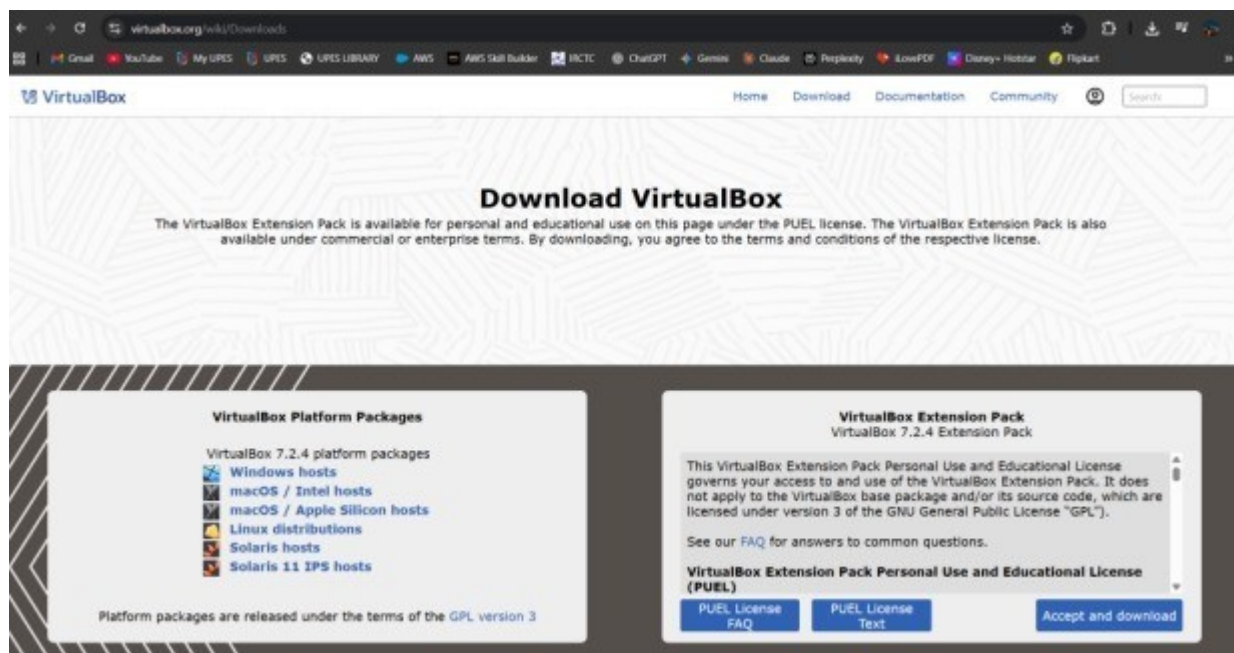
Characteristics:

- Shared kernel
- Lightweight
- Fast startup
- Efficient resource usage

3 Experiment Setup – Part A: Virtual Machine (Windows)

Step 1: Install VirtualBox

- Download VirtualBox from the official website.
- Run the installer with default settings.
- Restart the system if prompted.

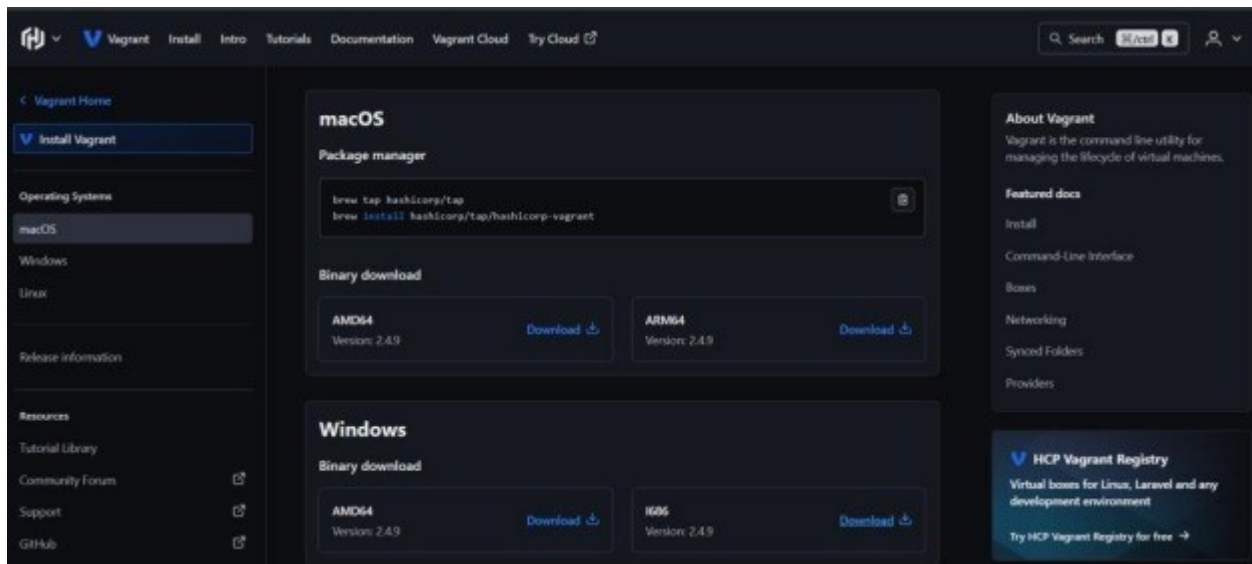


Step 2: Install Vagrant

- Download Vagrant for Windows.
- Install using default settings.

Verify installation:

```
vagrant --version
```



Step 3: Create Ubuntu VM using Vagrant

Initialize Vagrant:

```
vagrant init
```

```
hashicorp/bionic64 Start the
```

VM: `vagrant up`

Explanation:

`vagrant init hashicorp/bionic64` creates a Vagrantfile in the project folder.

`hashicorp/bionic64` refers to Ubuntu 18.04 LTS (64-bit).

`vagrant up` reads the Vagrantfile, downloads the image (if needed), allocates CPU/RAM, and boots the VM.

Access the VM:

```
vagrant ssh
```

This connects to the VM via SSH without requiring a password.

Step 4: Install Nginx inside VM

```
sudo apt update  
sudo apt install -y nginx  
sudo systemctl start nginx
```

Measure startup time:

```
time systemctl start nginx
```

Step 5: Verify Nginx

```
curl localhost
```

Stop and remove VM:

```
vagrant halt  
vagrant destroy
```

4 Experiment Setup – Part B: Containers using WSL

Step 1: Install WSL 2

```
wsl --install
```

Verify installation:

```
wsl --version
```

```
PS C:\WINDOWS\system32> wsl --version
WSL version: 2.6.3.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.71
MSRDC version: 1.2.6353
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.7462
PS C:\WINDOWS\system32> |
```

Step 2: Install Ubuntu on WSL

```
wsl --install -d
```

Ubuntu Verify: `wsl -l -v`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\cloud sem 5\LAB sem6\lab 1> wsl -l -v
  NAME                STATE      VERSION
* Ubuntu-24.04        Stopped    2
PS D:\cloud sem 5\LAB sem6\lab 1> wsl
```

Step 3: Install Docker Engine inside WSL

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Verify Docker:

```
docker --version
```

```

Created symlink /etc/systemd/system/sockets.target.wants/docker.socket →
/usr/lib/systemd/system/docker.socket
Processing triggers for man-db (2.12.043) ...
Processing triggers for libc-bin (2.39-0ubuntu8.5) ...
Docker version 29.1.5, build 06efee6
AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/ docker --version
Docker version 29.1.5, build 06efee6
AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/ sudo apt install podman -y

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$
Podman --version
Docker version 29.1.5, build 06efee6

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$
Reading package lists... Done
Building dependency tree.. Done
Reading state information.. Done
The following additional packages will be installed:
  aardvark-dns buildah catatonit common containernetworking-plugins fuse-overlayfs
  golang-github-containers-common golang-github-containers-image libgpgme11t64 libsudi4
  netav
  passt uidmap

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$

```

Step 4: Run Nginx Container

Pull image:

docker pull ubuntu Run container: docker run -d -p

8080:80 --name nginx-container nginx

```

tainer nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befee2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
390da5d749864f89cd67e67633131a2e0350b5f4f473878817ecb627fc0a983c

```

Explanation:

`docker pull ubuntu` downloads the Ubuntu image from Docker Hub.

`docker run -d -p8080:80--name nginx-containernginx` runs Nginx in detached mode and

maps port 8080 to container port 80.

Step 5: Verify Nginx in Container

`curl localhost:8080`

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
```

5 Resource Utilization Observation

5.1 VM Observation Commands

Memory usage:

`free -h`

Real-time monitoring:


```

2[ 0.0%] 3[ 0.1%] 0[ 0.0%] 9[
2[ 0.4%] 3[ 0.6%] 7[ 0.0%] 12[
3[ 0.4%] 3[ 0.4%] 8[ 0.0%] 11[
Mem[|||||]
Sup[
0.90/7.66G Tasks: 45, 66 thr, 0 kthr; 2 running
066/2.00 Load average: 0.11 0.12 0.07
Uptime: 00:21:18

P(%) PR VIRT RES SHR S CPU+AWM FINE: Command:
1 1 0 22260 13606 300A S 0.0 0.1 0.666 /sbin/init
1 0 0 2140 6308 4374 S 0.0 0.0 0.096 /sbin/init
1 0 20 6130 1290 1008 S 0.0 0.1 0.008 plant: ---control-socket 7 --log-level 4 --server-fd 8 --log-truncate
1 0 20 3720 1997 1540 S 0.0 0.0 0.096 pesor: ---control-socket 7 --log-level 4 --server-fd 8 --log-truncate
1 0 20 2120 1341 1859 S 0.0 1.6 0.090 finit
3 8 20 22590 21608 23198 0 0.0 0.0 0.528 /usr/lib/systemd/systemd-journal
3 sevd/76 13790 6403 1095 S 0.0 0.1 0.664 /usr/lib/systemd/systemd-eectl
3 sevd/76 15920 29593 35069 S 0.0 0.1 0.000 /usr/lib/systemd/systemd-recolved
1 rsyslog 12948 1210 9640 S 0.0 0.1 0.001 /usr/lib/systemd/systemd-tikaeytd
2 messagebus 7790 1934 1800 S 0.0 0.1 0.099 /usr/lib/systemd/systemd-messaging
1 raynsq 9940 2363 8648 S 0.0 0.0 0.006 /usr/lib/systemd/systemd-mayned
7 rayslo 2200 2600 2543 S 0.0 0.2 0.601 0hux-darned --seectem --adorase-systemd: --nefork --npidfile --syslog*00 --syslog-only
6 rsyslog 11236 31028 17632 S 0.0 0.1 0.001 /usr/lib/systemd/systemd-logand
8 rsyslog 2680 3268 1886 S 0.0 0.1 0.002 /usr/lib/systemd/systemd-ont -v0
6 mapag 5838 2368 6685 S 0.0 0.1 0.003 /usr/lib/entaisj --mc -p --neclear --keep-baud - 115200,38400,9600 vt220
14 root a 7943 13066 4830 S 0.0 0.1 0.001 /usr/librevdled -p -linux
11 root i 5700 63900 17359 S 0.0 0.2 0.034 /usr/lib//arobsa -p -c --infiloar - Linux
12 root 1 5700 11568 15286 S 0.0 0.1 0.008 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
13 Raghav/rck 1500 13995 10886 S 0.0 0.1 0.004 /usr/bin/polaib3-pen-service -vv
11 Raghavm 1500 11568 14796 S 0.0 0.1 0.004 /usr/bin/palaib3-pen-service -vv
14 Raghavm 1679 16796 9530 S 0.0 0.1 0.004 /usr/bin/pelkit3-pen-service -vv
11 Raghavm 1440 12966 15298 S 0.0 0.1 0.004 /usr/lib/pelkit3-pen-service -vv
11 rrtk 1504 11660 12590 S 0.0 0.1 0.004 /usr/lib/polkid3-pen-service -vv
16 root 1 2961 17080 18836 S 0.0 0.1 0.006 /usr/bin/pelkit3/bpt --m-adama
16 root 1 3154 13480 91578 S 0.0 0.1 0.004 /usr/lib/polkib/bpt --m-angne
16 root 1 1162 13480 69790 S 0.0 0.1 0.004 /usr/lib/polkib/bpt --m-uebng
19 root 1 7780 13480 33548 S 0.0 0.1 0.004 /usr/bin/polkib/bpt --m-uebng -vv
20 root 1 2380 31200 83799 S 0.0 0.1 0.004 /usr/lib/polkib/bpt --m-uebng -vv
07 Raghav/Malik BCDWIM: 85990 S 0.0 0.1 0.008 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
x0 Raghav/Malik BCDWIM: 12646 S 0.0 0.1 0.004 /usr/bin/containerd --usac
x9 root 1602 11200 16526 S 0.0 0.1 0.004 /-c-l-a -f
21 root 1545 11680 80230 S 0.0 0.1 0.004 /oppeh3
32 polkitd 1605 57583 69790 S 0.0 0.1 0.004 /usr/bn/polkit-1/polkitd --cdemg
34 polkitd 1605 57587 67792 S 0.0 0.2 0.004 /usr/bn/polkit-1/polkitd --leburg
35 polkitd 1605 57190 59498 S 0.0 0.1 0.004 /usr/bn/polkit-2/polkitd --leburg
1568 RAaghav/asolin 51500 59180 S 0.0 0.1 0.666 /usr/bin/containerd --debg
1516 poltica 1609 60093 69958 S 0.0 0.1 0.004 /usr/bn/polkit-3/polkitd --debg
1511 polkitd 1600 55886 68772 S 0.0 0.1 0.004 /usr/bn/polkit-1/polkitd --debg
1522 poltica 1601 77082 69770 S 0.0 0.1 0.004 /usr/bn/polkit-3/polkitd --debg
1515 conpa 1509 61500 73796 S 0.0 0.1 0.004 /usr/bin/containerd
1563 conda-1 1210 57082 66875 S 0.0 0.1 0.004 /sytex4

F1=Help F2=Setup F3=Search F4=Filter F5=Tree F6=SortBy F7=File F8=Kill F9=Quit

```

systemd-analyze

```

Startup finished in 2.815s (userspace)
graphical.target reached after 2.774s in userspace.

```

Container resource usage:

```
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
390da5d74986	nginx-container	0.00%	10.32MiB / 7.611GiB	0.13%	2.09kB / 1.4kB	0B / 12.3k

Memory usage:

free -h


```
[2]+  Stopped                  docker stats
aryanraj: .....-docker stats
aryanraj@B2CMN4K:/mnt/d/cloud sem 5/LAB

$$ free -h
      total        used        free      shared  buff/cache
Mem:    7.6Gi      606Mi      6.7Gi       3.7Mi       494Mi
Swap:   2.0Gi         0B      2.0Gi           7.0Gi

aryanraj@B2CMN4K:/mnt/d/cloud sem 5/LAB sem1
```

6 Comparison: Virtual Machine vs Container

Parameter	Virtual Machine	Container
Boot Time	Slower (Full OS boot)	Very Fast
RAM Usage	High	Low
CPU Overhead	Higher	Minimal
Disk Usage	Large	Lightweight
Isolation	Strong	Moderate

7 Conclusion

Virtual Machines provide strong isolation and are suitable for legacy systems and full OS environments.

Containers are lightweight, start quickly, consume fewer resources, and are ideal for modern DevOps workflows and microservices architecture.